AIA6: Fine-Tuning and Custom Models

Module Introduction and Theoretical Foundations

Fine-tuning transforms generalist LLMs into **specialized domain experts** while Parameter-Efficient Fine-Tuning (PEFT) methods enable customization at 0.01-1% parameter cost. This module covers the complete pipeline from data preparation to production deployment, grounded in **information geometry**, **catastrophic forgetting theory**, and **scaling laws for adaptation**.

Key Theoretical Insights:

1. **Task Arithmetic**: Linear combinations in weight space
2. **Fisher Information**: Task interference measurement
3. **LoRA Singular Value Theory**: Low-rank adaptation optimality
4. **Continued Pretraining**: Bridge to instruction tuning

Method Hierarchy:

text

Full Fine-tuning → LoRA → QLoRA → Prompt Tuning → In-Context Learning

Cost: 100% → 0.1% → 0.01% → 0%

(Figure 1: PEFT methods vs full FT - accuracy/cost tradeoff)

Section 1: Dataset Preparation and Quality Engineering

Data Pipeline Theory:

text

Quality > Quantity (Xie 2023)

DPO prefers high-quality preference pairs

Instruction Data Formats:

text

Alpaca: {"instruction", "input", "output"}

ShareGPT: ChatML [{"role": "user/system/assistant", "content"}]

Data Processing Pipeline:

python

```python
import datasets

from torch.utils.data import Dataset

import json
```

```python
class InstructionDataset(Dataset):
    def __init__(self, data_path, tokenizer, max_length=2048):
        self.data = self.load_jsonl(data_path)
        self.tokenizer = tokenizer
        self.max_length = max_length

    def load_jsonl(self, path):
        data = []
        with open(path) as f:
            for line in f:
                data.append(json.loads(line))
        return data

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data[idx]

        # Format as chat
        messages = [{"role": "system", "content": item.get("system", "")},
                    {"role": "user", "content": item["instruction"]},
                    {"role": "assistant", "content": item["output"]}]

        text = self.tokenizer.apply_chat_template(messages, tokenize=False)

        # Tokenize with truncation
        tokenized = self.tokenizer(
            text, truncation=True, max_length=self.max_length,
            padding="max_length", return_tensors="pt"
        )
```

```
        return {
            "input_ids": tokenized.input_ids.squeeze(),
            "attention_mask": tokenized.attention_mask.squeeze(),
            "labels": tokenized.input_ids.squeeze()  # Self-supervised
        }

# Quality filtering
def filter_quality(data, min_length=10, max_length=2048, dedup=True):
    filtered = []
    seen = set()

    for item in data:
        text = item['instruction'] + item['output']
        if (min_length <= len(text) <= max_length and
            (not dedup or text not in seen)):
            filtered.append(item)
            seen.add(text)

    return filtered
```

Synthetic Data Generation:

text

Self-Instruct (Wang 2022): LLM generates instructions

Orion (Mitra 2024): Iterative refinement

Section 2: LoRA and PEFT Methods - Mathematical Foundations

Low-Rank Adaptation (LoRA, Hu 2021):

text

$W = W_o + \Delta W = W_o + B \cdot A$

$B \in \mathbb{R}^{(d \times r)}$, $A \in \mathbb{R}^{(r \times k)}$, $r << \min(d,k)$

Train only B,A (0.1% params)

Theoretical Justification:

text

Tasks occupy low-dimensional subspaces

Singular values decay rapidly → low r sufficient

QLoRA (Dettmers 2023) - Consumer GPU Fine-tuning:

1. **4-bit NormalFloat (NF4)** quantization
2. **Double quantization** (quantize quantization constants)
3. **Paged AdamW** optimizer

Complete LoRA Implementation:

python

```python
from peft import LoraConfig, get_peft_model, TaskType
import bitsandbytes as bnb

def setup_lora_model(base_model, r=16, lora_alpha=32, target_modules=None):
    if target_modules is None:
        target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                          "gate_proj", "up_proj", "down_proj"]

    config = LoraConfig(
        r=r,
        lora_alpha=lora_alpha,
        target_modules=target_modules,
        lora_dropout=0.05,
        bias="none",
        task_type=TaskType.CAUSAL_LM
    )

    model = get_peft_model(base_model, config)
    model.print_trainable_parameters()
    return model
```

```python
# QLoRA setup (4-bit base)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b-hf",
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    device_map="auto"
)

lora_model = setup_lora_model(model)
```

Section 3: Training Recipes and Hyperparameters

SFT (Supervised Fine-Tuning) Recipe:

```text
Dataset: 10k-100k high-quality examples
Batch: 32-256 (gradient accumulation)
LR: 1e-5 → 5e-5
Warmup: 10%
Cosine decay
Epochs: 1-3
```

Complete Training Loop:

```python
from transformers import TrainingArguments, Trainer
from trl import SFTTrainer

training_args = TrainingArguments(
    output_dir="./lora-sft",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=8,
    learning_rate=2e-4,
    num_train_epochs=2,
```

```
    logging_steps=10,
    save_steps=500,
    evaluation_strategy="steps",
    eval_steps=500,
    warmup_steps=100,
    lr_scheduler_type="cosine",
    fp16=True,
    report_to="wandb",
    remove_unused_columns=False,
)

trainer = SFTTrainer(
    model=lora_model,
    tokenizer=tokenizer,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    dataset_text_field="text",  # or processing_func
    max_seq_length=2048,
    args=training_args,
    packing=True,  # Pack sequences
)

trainer.train()
```

DPO (Direct Preference Optimization):

text

Skip RL: Direct policy optimization

L_DPO = -log σ(β log(π(y_w|x)/π_ref) - log(π(y_l|x)/π_ref))

Section 4: Domain Adaptation Strategies

Continued Pretraining:

text

Phase 1: Domain corpus (no labels)

Phase 2: Instruction tuning

Optimal: 1B domain tokens

Multi-Task Learning:

text

Share trunk, task-specific LoRAs

Merge via task vectors (Ilharco 2023)

Task Arithmetic:

text

Δtask = W_task - W_base

W_new = W_base + α Δtask_finance + β Δtask_legal

MergeKit Implementation:

python

```python
# SLERP merging (linear interpolation in hyperspheres)
from mergekit.merge import merge_slerp

merge_config = {
    "models": [
        "base_model",
        "finance_lora",
        "legal_lora"
    ],
    "parameters": {
        "t": [0, 0.5, 0.5],
        "weight": [1, 1, 1]
    }
}
```

Section 5: Advanced PEFT Methods

DoRA (Orthogonal LoRA):

text

Decompose magnitude + direction

ΔW = magnitude · direction

VeRA (Rank-One Updates):

text

Single vector per layer (0.01% params)

Mixture of LoRAs:

text

Route to task-specific adapters

SwitchTransformer-style gating

Section 6: Evaluation and Production Deployment

Fine-tuning Evaluation Framework:

text

1. Perplexity (language modeling)
2. Task-specific metrics
3. Catastrophic forgetting (EWC penalty)
4. Robustness (adversarial examples)

Production Pipeline:

python

```python
class ProductionFT:
    def __init__(self, base_model_path):
        self.model = self.load_quantized(base_model_path)

    def train_specialist(self, dataset_path, task_name):
        # LoRA training
        adapter_path = f"adapters/{task_name}"
        self.train_lora(dataset_path, adapter_path)
        return adapter_path

    @torch.no_grad()
    def infer(self, prompt, adapter_path=None):
```

```
        if adapter_path:
            self.model.load_adapter(adapter_path)
        return self.model.generate(prompt)

    def deploy(self, adapter_path, endpoint_name):
        # vLLM/TGI deployment
        pass
```

Monitoring:

text

Drift detection: KL(p_new || p_base)

Performance regression alerting

Section 7: Case Studies and Capstone Projects

Case Study: Med-PaLM 2

text

Continued pretraining: PubMed

Instruction: 100k medical Q&A

RLHF: Clinician preferences

Result: > SOTA closed-source

Capstone Project: Custom Domain LLM

text

1. Domain corpus → continued pretraining
2. Synthetic instruction data
3. Multi-LoRA specialists
4. Ensemble deployment